

Backend Schema & Database Design Document

Hospital Bed Allocation System

Version: 1.0
Date: 2026-05-17
Database: MySQL 8.0+
Design Standard: 3NF / BCNF with full referential integrity

1. Database Overview

1.1 Database Name

hospital_bed_db

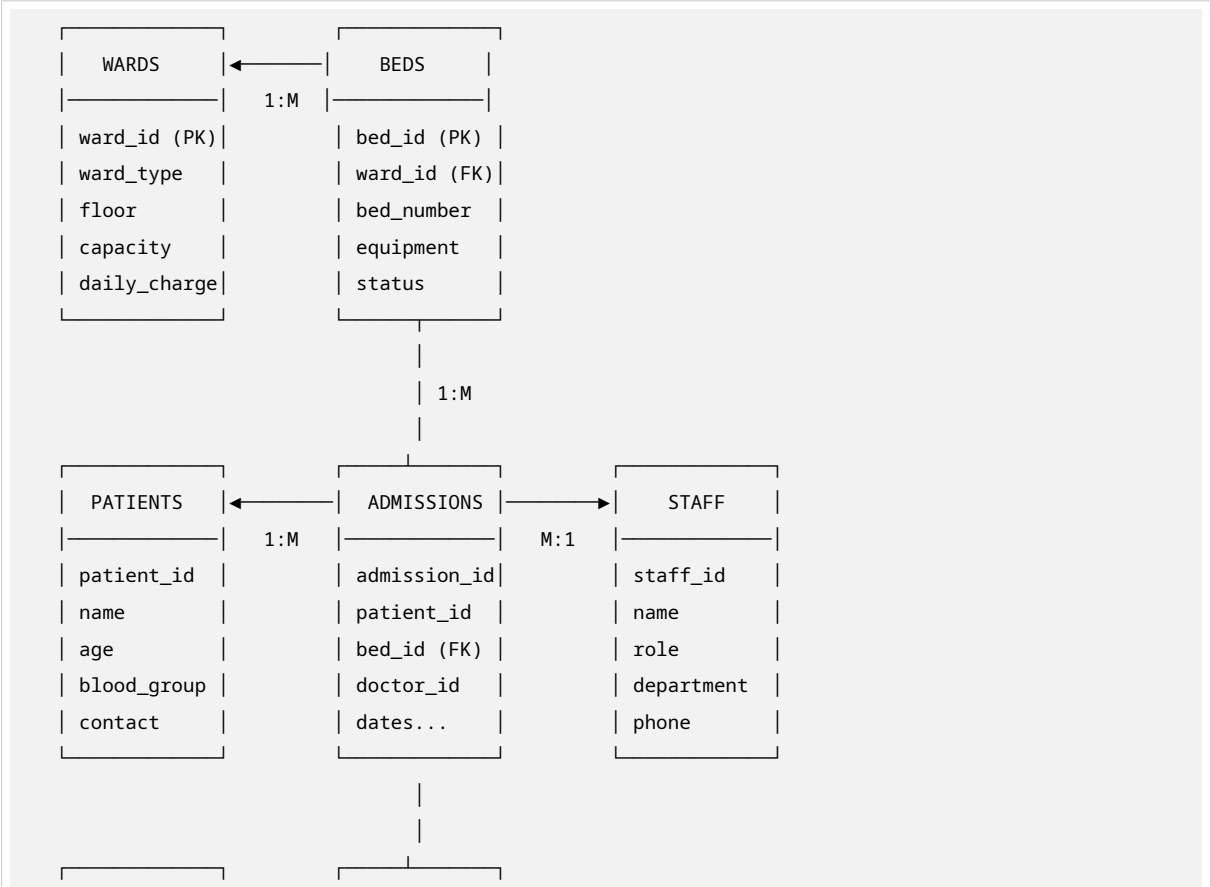
1.2 Character Set

utf8mb4 with utf8mb4_unicode_ci collation

1.3 Storage Engine

InnoDB (required for transactions, foreign keys, and row-level locking)

2. Entity-Relationship Diagram (Logical)



WAITLIST	BED_STATUS_
	AUDIT
waitlist_id	
patient_id	audit_id
ward_type	bed_id (FK)
priority	old_status
request_time	new_status

BED_MAINTENANCE_LOG
log_id (PK)
bed_id (FK)
start_time
end_time
reason
resolved_by (FK)

Cardinality Summary:

- Ward → Bed: **1:N** (One ward has many beds)
- Bed → Admission: **1:N** (One bed has many admissions over time, but only one active at a time)
- Patient → Admission: **1:N** (One patient can have multiple admissions historically)
- Staff → Admission: **1:N** (One doctor can handle many active admissions)
- Bed → BedMaintenanceLog: **1:N** (One bed can have multiple maintenance records)
- Patient → Waitlist: **1:N** (One patient can be waitlisted multiple times historically)

3. Complete Schema (DDL)

3.1 Database Creation

```
-- Create database with proper character set
CREATE DATABASE IF NOT EXISTS hospital_bed_db
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE hospital_bed_db;
```

3.2 Wards Table

```
CREATE TABLE wards (
    ward_id          INT PRIMARY KEY AUTO_INCREMENT,
    ward_type        VARCHAR(20) NOT NULL,
    floor            INT NOT NULL,
    capacity          INT NOT NULL,
    daily_charge      DECIMAL(10,2) NOT NULL,
```

```

-- CHECK constraints enforce domain integrity
CONSTRAINT chk_ward_type CHECK (ward_type IN ('General','ICU','Emergency','Private')),
CONSTRAINT chk_ward_floor CHECK (floor > 0 AND floor <= 20),
CONSTRAINT chk_ward_capacity CHECK (capacity > 0 AND capacity <= 100),
CONSTRAINT chk_ward_charge CHECK (daily_charge >= 0),

-- Unique constraint: No duplicate ward types on same floor
CONSTRAINT uq_ward_type_floor UNIQUE (ward_type, floor)
) ENGINE=InnoDB;

```

Functional Dependencies:

- ward_id → ward_type, floor, capacity, daily_charge

Normal Form: BCNF — ward_id is the only determinant and it is the primary key.

3.3 Beds Table

```

CREATE TABLE beds (
    bed_id          INT PRIMARY KEY AUTO_INCREMENT,
    ward_id         INT NOT NULL,
    bed_number      VARCHAR(10) NOT NULL,
    equipment_status VARCHAR(100) DEFAULT 'Standard',
    current_status  VARCHAR(20) DEFAULT 'Available',

    -- Foreign key to wards
    CONSTRAINT fk_bed_ward FOREIGN KEY (ward_id)
        REFERENCES wards(ward_id)
        ON DELETE RESTRICT      -- Cannot delete ward if beds exist
        ON UPDATE CASCADE,     -- Ward ID updates propagate

    -- Domain constraints
    CONSTRAINT chk_bed_status CHECK (current_status IN ('Available','Occupied','Maintenance')),
    CONSTRAINT chk_equipment CHECK (equipment_status IS NOT NULL),

    -- Unique: Bed number must be unique within a ward
    CONSTRAINT uq_bed_ward_number UNIQUE (ward_id, bed_number)
) ENGINE=InnoDB;

```

Functional Dependencies:

- bed_id → ward_id, bed_number, equipment_status, current_status
- (ward_id, bed_number) → bed_id, equipment_status, current_status (candidate key)

Normal Form: BCNF

3.4 Patients Table

```

CREATE TABLE patients (
    patient_id      INT PRIMARY KEY AUTO_INCREMENT,
    name            VARCHAR(100) NOT NULL,
    age             INT,

```

```

blood_group      VARCHAR(5),
contact          VARCHAR(15) NOT NULL,
address          VARCHAR(255),
emergency_contact VARCHAR(15),
registered_date   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

-- Domain constraints
CONSTRAINT chk_patient_age CHECK (age >= 0 AND age <= 150),
CONSTRAINT chk_patient_name CHECK (CHAR_LENGTH(name) >= 2),
CONSTRAINT chk_blood_group CHECK (blood_group IN ('A+', 'A-', 'B+', 'B-', 'O+', 'O-', 'AB+', 'AB-') OR
    ⇨ blood_group IS NULL),
CONSTRAINT chk_patient_contact CHECK (contact REGEXP '^[0-9]{10}$'),

-- Unique contact to prevent duplicate registrations
CONSTRAINT uq_patient_contact UNIQUE (contact)
) ENGINE=InnoDB;

```

Functional Dependencies:

- patient_id → name, age, blood_group, contact, address, emergency_contact, registered_date
- contact → patient_id, name, age, blood_group, address, emergency_contact (candidate key via UNIQUE constraint)

Normal Form: BCNF

3.5 Staff Table

```

CREATE TABLE staff (
    staff_id INT PRIMARY KEY AUTO_INCREMENT,
    name     VARCHAR(100) NOT NULL,
    role     VARCHAR(20) NOT NULL,
    department VARCHAR(50),
    phone    VARCHAR(15),

    -- Domain constraints
    CONSTRAINT chk_staff_role CHECK (role IN ('Doctor', 'Nurse', 'Admin')),
    CONSTRAINT chk_staff_name CHECK (CHAR_LENGTH(name) >= 2),
    CONSTRAINT chk_staff_phone CHECK (phone REGEXP '^[0-9]{10}$' OR phone IS NULL)
) ENGINE=InnoDB;

```

Functional Dependencies:

- staff_id → name, role, department, phone

Normal Form: BCNF

3.6 Admissions Table

```

CREATE TABLE admissions (
    admission_id INT PRIMARY KEY AUTO_INCREMENT,
    patient_id   INT NOT NULL,
    bed_id       INT NOT NULL,
    doctor_id    INT,

```

```

admission_date    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
expected_discharge DATE,
actual_discharge  TIMESTAMP NULL,
status            VARCHAR(20) DEFAULT 'Active',
notes            TEXT,

-- Foreign keys
CONSTRAINT fk_adm_patient FOREIGN KEY (patient_id)
    REFERENCES patients(patient_id)
    ON DELETE RESTRICT
    ON UPDATE CASCADE,

CONSTRAINT fk_adm_bed FOREIGN KEY (bed_id)
    REFERENCES beds(bed_id)
    ON DELETE RESTRICT
    ON UPDATE CASCADE,

CONSTRAINT fk_adm_doctor FOREIGN KEY (doctor_id)
    REFERENCES staff(staff_id)
    ON DELETE SET NULL      -- If doctor deleted, admission record preserved
    ON UPDATE CASCADE,

-- Domain constraints
CONSTRAINT chk_adm_status CHECK (status IN ('Active', 'Discharged', 'Transferred')),
CONSTRAINT chk_adm_dates CHECK (expected_discharge IS NULL OR expected_discharge >=
    → DATE(admission_date)),

-- Critical constraint: A bed can have only ONE active admission at a time
-- This is implemented via a UNIQUE constraint on (bed_id, status) where status='Active'
-- Since MySQL doesn't support partial unique indexes directly, we use a trigger
-- to enforce this (see Trigger section below)

INDEX idx_adm_status (status),
INDEX idx_adm_bed_status (bed_id, status)
) ENGINE=InnoDB;
```

Functional Dependencies:

- admission_id → patient_id, bed_id, doctor_id, admission_date, expected_discharge, actual_discharge, status, notes
- (bed_id, status='Active') → admission_id, patient_id, doctor_id (functional dependency enforced by application logic + trigger)

Normal Form: BCNF

3.7 Waitlist Table

```

CREATE TABLE waitlist (
    waitlist_id    INT PRIMARY KEY AUTO_INCREMENT,
    patient_id     INT NOT NULL,
    requested_ward_type VARCHAR(20) NOT NULL,
    priority_score  INT NOT NULL DEFAULT 0,
```

```

request_time      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
status            VARCHAR(20) DEFAULT 'Waiting',
notes             TEXT,

-- Foreign key
CONSTRAINT fk_wl_patient FOREIGN KEY (patient_id)
    REFERENCES patients(patient_id)
    ON DELETE CASCADE      -- Remove waitlist entry if patient deleted
    ON UPDATE CASCADE,

-- Domain constraints
CONSTRAINT chk_wl_status CHECK (status IN ('Waiting', 'Allocated', 'Cancelled')),
CONSTRAINT chk_wl_ward CHECK (requested_ward_type IN ('General', 'ICU', 'Emergency', 'Private')),
CONSTRAINT chk_wl_priority CHECK (priority_score >= 0),

-- A patient should not be waitlisted twice for the same ward type simultaneously
CONSTRAINT uq_wl_patient_ward UNIQUE (patient_id, requested_ward_type, status)
) ENGINE=InnoDB;

```

Functional Dependencies:

- waitlist_id → patient_id, requested_ward_type, priority_score, request_time, status, notes

Normal Form: BCNF

3.8 Bed Maintenance Log Table

```

CREATE TABLE bed_maintenance_log (
    log_id          INT PRIMARY KEY AUTO_INCREMENT,
    bed_id          INT NOT NULL,
    start_time      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    end_time        TIMESTAMP NULL,
    reason          VARCHAR(255) NOT NULL,
    resolved_by     INT,
    status          VARCHAR(20) DEFAULT 'Ongoing',

-- Foreign keys
CONSTRAINT fk_maint_bed FOREIGN KEY (bed_id)
    REFERENCES beds(bed_id)
    ON DELETE CASCADE
    ON UPDATE CASCADE,

CONSTRAINT fk_maint_staff FOREIGN KEY (resolved_by)
    REFERENCES staff(staff_id)
    ON DELETE SET NULL
    ON UPDATE CASCADE,

-- Domain constraints
CONSTRAINT chk_maint_status CHECK (status IN ('Ongoing', 'Completed', 'Cancelled')),
CONSTRAINT chk_maint_dates CHECK (end_time IS NULL OR end_time >= start_time),
CONSTRAINT chk_maint_reason CHECK (CHAR_LENGTH(reason) >= 3)
) ENGINE=InnoDB;

```

Functional Dependencies:

- log_id → bed_id, start_time, end_time, reason, resolved_by, status

Normal Form: BCNF

3.9 Bed Status Audit Table

```
CREATE TABLE bed_status_audit (  
  audit_id      INT PRIMARY KEY AUTO_INCREMENT,  
  bed_id        INT NOT NULL,  
  old_status    VARCHAR(20),  
  new_status    VARCHAR(20) NOT NULL,  
  changed_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  changed_by    VARCHAR(50),  
  reason        VARCHAR(255),  
  
  -- Foreign key  
  CONSTRAINT fk_audit_bed FOREIGN KEY (bed_id)  
    REFERENCES beds (bed_id)  
    ON DELETE CASCADE  
    ON UPDATE CASCADE,  
  
  -- Domain constraints  
  CONSTRAINT chk_audit_old CHECK (old_status IN ('Available','Occupied','Maintenance') OR old_status  
    ⇨ IS NULL),  
  CONSTRAINT chk_audit_new CHECK (new_status IN ('Available','Occupied','Maintenance'))  
) ENGINE=InnoDB;
```

Functional Dependencies:

- audit_id → bed_id, old_status, new_status, changed_at, changed_by, reason

Normal Form: BCNF

4. Normalization Analysis

4.1 Unnormalized Form (UNF)

Monster Table: hospital_records

patient_id	patient_name	age	ward_type	bed_no	bed_equipment	doctor_name	nurse_name	admission_date	discharge_date	medication
P101	Rahul	45	ICU	B12	Ventilator, Monitor	Dr. Sharma, Dr. Patel	Nurse A, Nurse B	2026-05-10	2026-05-15	Paracetamol, Antibiotic

Anomalies in UNF:

- Insertion Anomaly:** Cannot add a new bed without assigning a patient.
- Deletion Anomaly:** Deleting a patient loses bed and ward information.

- **Update Anomaly:** Changing ward floor requires updating all bed records in that ward.
- **Multi-valued attributes:** medicines, doctor_name, nurse_name, bed_equipment contain comma-separated values.

4.2 First Normal Form (1NF)

Rules Applied:

- All attributes contain only atomic (indivisible) values.
- No repeating groups.

Decomposition:

- patients(patient_id, name, age, address, contact)
- beds(bed_id, ward_type, floor, bed_number, equipment_status)
- admissions(admission_id, patient_id, bed_id, admission_date, discharge_date)
- staff(staff_id, name, role, department)
- admission_staff(admission_id, staff_id, role_in_admission) — resolves M:N
- medicines(medicine_id, name, dosage)
- prescriptions(admission_id, medicine_id, dosage, time)

Remaining Issues:

- Partial dependencies exist (e.g., bed_id → ward_type, floor but admission_id is the primary key of admissions).

4.3 Second Normal Form (2NF)

Rules Applied:

- Must be in 1NF.
- No partial dependencies (non-prime attributes must depend on the ENTIRE candidate key, not part of it).

Partial Dependencies Identified:

- In admissions with composite key (admission_id): bed_id → ward_type, floor, bed_number (bed attributes depend on bed_id, not admission_id).
- doctor_name → doctor_department (depends on doctor, not admission).

Decomposition:

- Extract wards(ward_id, ward_type, floor, capacity, daily_charge)
- Extract beds(bed_id, ward_id, bed_number, equipment_status, current_status)
- Extract staff(staff_id, name, role, department, phone)
- admissions now contains only admission-specific attributes.

4.4 Third Normal Form (3NF)

Rules Applied:

- Must be in 2NF.
- No transitive dependencies (non-prime attributes must depend directly on the candidate key, not on other non-prime attributes).

Transitive Dependencies Identified:

- $\text{ward_id} \rightarrow \text{ward_type} \rightarrow \text{daily_charge}$ (charge depends on ward_type, which depends on ward_id).
- Already resolved in 2NF by extracting wards table.
- $\text{patient_id} \rightarrow \text{address, contact}$ (contact info depends on patient, not admission).
- Already resolved by separate patients table.

Result: All tables in final schema are in 3NF.

4.5 Boyce-Codd Normal Form (BCNF)

Rules Applied:

- Must be in 3NF.
- For every functional dependency $X \rightarrow Y$, X must be a superkey.

Verification:

Table	FD	Left Side is Superkey?
wards	$\text{ward_id} \rightarrow \text{ward_type, floor, capacity, daily_charge}$	Yes —ward_id is PK
beds	$\text{bed_id} \rightarrow \text{ward_id, bed_number, equipment_status, current_status}$	Yes —bed_id is PK
patients	$\text{patient_id} \rightarrow \text{name, age, blood_group, contact, address}$	Yes —patient_id is PK
patients	$\text{contact} \rightarrow \text{patient_id, name, age...}$	Yes —contact is UNIQUE
staff	$\text{staff_id} \rightarrow \text{name, role, department, phone}$	Yes —staff_id is PK
admissions	$\text{admission_id} \rightarrow \text{patient_id, bed_id, doctor_id, dates, status}$	Yes —admission_id is PK
waitlist	$\text{waitlist_id} \rightarrow \text{patient_id, requested_ward_type, priority_score...}$	Yes —waitlist_id is PK
bed_maintenance_log	$\text{log_id} \rightarrow \text{bed_id, start_time, end_time, reason, resolved_by}$	Yes —log_id is PK
bed_status_audit	$\text{audit_id} \rightarrow \text{bed_id, old_status, new_status, changed_at...}$	Yes —audit_id is PK

Conclusion: All tables satisfy BCNF. No further decomposition needed.

5. Advanced DBMS Objects

5.1 Triggers

Trigger 1: Prevent Multiple Active Admissions Per Bed MySQL does not support partial unique indexes (e.g., UNIQUE on `bed_id` only where `status='Active'`). We enforce this via a trigger:

```
DELIMITER //
```

```
CREATE TRIGGER trg_prevent_double_active_admission
BEFORE INSERT ON admissions
FOR EACH ROW
BEGIN
    DECLARE v_active_count INT;

    -- Only check if the new admission is Active
    IF NEW.status = 'Active' THEN
        SELECT COUNT(*) INTO v_active_count
        FROM admissions
        WHERE bed_id = NEW.bed_id
        AND status = 'Active';

        IF v_active_count > 0 THEN
            SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Error: Bed is already occupied by an active admission';
        END IF;
    END IF;
END//
```

```
DELIMITER ;
```

Purpose: Acts as an **ASSERTION** equivalent — ensures the business rule “one active admission per bed” is enforced at the database level, not just application level.

Trigger 2: Audit Log on Bed Status Change

```
DELIMITER //
```

```
CREATE TRIGGER trg_audit_bed_status_update
AFTER UPDATE ON beds
FOR EACH ROW
BEGIN
    -- Only log when status actually changes
    IF OLD.current_status != NEW.current_status THEN
        INSERT INTO bed_status_audit (
            bed_id,
            old_status,
            new_status,
            changed_by,
```

```

        reason
    ) VALUES (
        NEW.bed_id,
        OLD.current_status,
        NEW.current_status,
        COALESCE(@session_user, 'SYSTEM'),
        COALESCE(@status_change_reason, 'Manual Update')
    );
END IF;
END//

DELIMITER ;

```

Purpose: Maintains complete audit trail for compliance and debugging.

Trigger 3: Auto-Update Bed Status on Admission Insert

```

DELIMITER //

CREATE TRIGGER trg_bed_occupied_on_admit
AFTER INSERT ON admissions
FOR EACH ROW
BEGIN
    IF NEW.status = 'Active' THEN
        UPDATE beds
        SET current_status = 'Occupied'
        WHERE bed_id = NEW.bed_id;
    END IF;
END//

DELIMITER ;

```

Purpose: Keeps beds.current_status synchronized with admissions table automatically.

Trigger 4: Auto-Update Bed Status on Discharge

```

DELIMITER //

CREATE TRIGGER trg_bed_available_on_discharge
AFTER UPDATE ON admissions
FOR EACH ROW
BEGIN
    IF OLD.status = 'Active' AND NEW.status = 'Discharged' THEN
        -- Set context variable for audit trigger
        SET @status_change_reason = 'Patient Discharge';

        UPDATE beds
        SET current_status = 'Available'
        WHERE bed_id = NEW.bed_id;

        SET @status_change_reason = NULL;
    END IF;
END//

```

```

END//

DELIMITER ;

```

Purpose: Frees the bed automatically when a patient is discharged.

Trigger 5: Prevent Maintenance Start on Occupied Bed

```

DELIMITER //

CREATE TRIGGER trg_prevent_maint_occupied
BEFORE INSERT ON bed_maintenance_log
FOR EACH ROW
BEGIN
    DECLARE v_bed_status VARCHAR(20);

    SELECT current_status INTO v_bed_status
    FROM beds
    WHERE bed_id = NEW.bed_id;

    IF v_bed_status = 'Occupied' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: Cannot start maintenance on an occupied bed';
    END IF;
END//

DELIMITER ;

```

Purpose: Business rule enforcement at DB level.

5.2 Stored Procedures

Procedure 1: Allocate Bed (Atomic Transaction) Uses **CURSOR** to iterate through waitlist if needed:

```

DELIMITER //

CREATE PROCEDURE sp_allocate_bed(
    IN p_patient_id INT,
    IN p_ward_type VARCHAR(20),
    IN p_doctor_id INT,
    IN p_expected_discharge DATE,
    IN p_notes TEXT,
    OUT p_bed_id INT,
    OUT p_admission_id INT,
    OUT p_success BOOLEAN,
    OUT p_message VARCHAR(255)
)
BEGIN
    DECLARE v_bed_id INT DEFAULT NULL;
    DECLARE v_admission_id INT DEFAULT NULL;
    DECLARE v_waitlist_patient_id INT;

```

```

DECLARE v_waitlist_id INT;
DECLARE v_waitlist_found BOOLEAN DEFAULT FALSE;

-- Cursor for checking waitlist (demonstrating CURSOR concept)
DECLARE waitlist_cursor CURSOR FOR
    SELECT waitlist_id, patient_id
    FROM waitlist
    WHERE requested_ward_type = p_ward_type
        AND status = 'Waiting'
    ORDER BY priority_score DESC, request_time ASC
    LIMIT 1;

DECLARE CONTINUE HANDLER FOR NOT FOUND
    SET v_waitlist_found = FALSE;

-- Start transaction
START TRANSACTION;

-- Step 1: Find first available bed in requested ward
SELECT b.bed_id INTO v_bed_id
FROM beds b
JOIN wards w ON b.ward_id = w.ward_id
WHERE w.ward_type = p_ward_type
    AND b.current_status = 'Available'
    AND b.equipment_status != 'Broken'
ORDER BY b.bed_id
LIMIT 1
FOR UPDATE; -- Pessimistic lock to prevent race conditions

IF v_bed_id IS NOT NULL THEN
    -- Step 2: Create admission
    INSERT INTO admissions (
        patient_id, bed_id, doctor_id,
        expected_discharge, status, notes
    ) VALUES (
        p_patient_id, v_bed_id, p_doctor_id,
        p_expected_discharge, 'Active', p_notes
    );

    SET v_admission_id = LAST_INSERT_ID();

    -- Step 3: Update bed status (trigger will also do this, but explicit for clarity)
    UPDATE beds
    SET current_status = 'Occupied'
    WHERE bed_id = v_bed_id;

    -- Step 4: Check if this patient was on waitlist and update
    UPDATE waitlist
    SET status = 'Allocated'
    WHERE patient_id = p_patient_id
        AND requested_ward_type = p_ward_type

```

```

        AND status = 'Waiting';

-- Commit
COMMIT;

SET p_bed_id = v_bed_id;
SET p_admission_id = v_admission_id;
SET p_success = TRUE;
SET p_message = CONCAT('Patient admitted to Bed ID: ', v_bed_id);

ELSE
    -- No bed available — add to waitlist
    INSERT INTO waitlist (
        patient_id, requested_ward_type, priority_score, status, notes
    ) VALUES (
        p_patient_id, p_ward_type,
        100, -- Default priority, application should calculate: severity * 100 + wait_hours
        'Waiting',
        p_notes
    );

    COMMIT;

    SET p_bed_id = NULL;
    SET p_admission_id = NULL;
    SET p_success = FALSE;
    SET p_message = 'No bed available. Patient added to waitlist.';
END IF;
END//

DELIMITER ;

```

Procedure 2: Discharge Patient with Waitlist Auto-Allocation Uses **CURSOR** to process waitlist after discharge:

```

DELIMITER //

CREATE PROCEDURE sp_discharge_patient(
    IN p_admission_id INT,
    IN p_maintenance_needed BOOLEAN,
    IN p_maintenance_reason VARCHAR(255),
    OUT p_message VARCHAR(255)
)
BEGIN
    DECLARE v_bed_id INT;
    DECLARE v_ward_id INT;
    DECLARE v_ward_type VARCHAR(20);
    DECLARE v_patient_name VARCHAR(100);
    DECLARE v_waitlist_patient_id INT;
    DECLARE v_waitlist_id INT;
    DECLARE v_new_admission_id INT;

```

```

DECLARE v_auto_allocated BOOLEAN DEFAULT FALSE;
DECLARE v_waitlist_found BOOLEAN DEFAULT TRUE;

-- Cursor to find highest priority waitlisted patient
DECLARE waitlist_cursor CURSOR FOR
    SELECT w.waitlist_id, w.patient_id
    FROM waitlist w
    JOIN patients p ON w.patient_id = p.patient_id
    WHERE w.requested_ward_type = v_ward_type
        AND w.status = 'Waiting'
    ORDER BY w.priority_score DESC, w.request_time ASC
    LIMIT 1;

DECLARE CONTINUE HANDLER FOR NOT FOUND
    SET v_waitlist_found = FALSE;

-- Start transaction
START TRANSACTION;

-- Get admission, bed, and ward details
SELECT
    a.bed_id, b.ward_id, w.ward_type, p.name
INTO
    v_bed_id, v_ward_id, v_ward_type, v_patient_name
FROM admissions a
JOIN beds b ON a.bed_id = b.bed_id
JOIN wards w ON b.ward_id = w.ward_id
JOIN patients p ON a.patient_id = p.patient_id
WHERE a.admission_id = p_admission_id
    AND a.status = 'Active';

IF v_bed_id IS NULL THEN
    ROLLBACK;
    SET p_message = 'Error: Active admission not found';
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Active admission not found';
END IF;

-- Update admission record
UPDATE admissions
SET status = 'Discharged',
    actual_discharge = NOW()
WHERE admission_id = p_admission_id;

IF p_maintenance_needed THEN
    -- Mark bed for maintenance
    SET @status_change_reason = 'Post-Discharge Maintenance';

    UPDATE beds
    SET current_status = 'Maintenance'
    WHERE bed_id = v_bed_id;

```

```

INSERT INTO bed_maintenance_log (
    bed_id, reason, status
) VALUES (
    v_bed_id,
    COALESCE(p_maintenance_reason, 'Post-discharge cleaning'),
    'Ongoing'
);

SET @status_change_reason = NULL;

COMMIT;

SET p_message = CONCAT(v_patient_name, ' discharged. Bed marked for maintenance.');
```

ELSE

```

-- Make bed available and check waitlist
UPDATE beds
SET current_status = 'Available'
WHERE bed_id = v_bed_id;

-- Open cursor to find waitlisted patient
OPEN waitlist_cursor;
FETCH waitlist_cursor INTO v_waitlist_id, v_waitlist_patient_id;
CLOSE waitlist_cursor;

IF v_waitlist_found AND v_waitlist_id IS NOT NULL THEN
    -- Auto-allocate to waitlisted patient
    UPDATE waitlist
    SET status = 'Allocated'
    WHERE waitlist_id = v_waitlist_id;

    INSERT INTO admissions (
        patient_id, bed_id, status, admission_date, notes
    ) VALUES (
        v_waitlist_patient_id, v_bed_id, 'Active', NOW(),
        'Auto-allocated from waitlist'
    );

    SET v_new_admission_id = LAST_INSERT_ID();
    SET v_auto_allocated = TRUE;
END IF;

COMMIT;

IF v_auto_allocated THEN
    SET p_message = CONCAT(
        v_patient_name,
        ' discharged. Bed auto-allocated to waitlisted patient.'
    );
ELSE
    SET p_message = CONCAT(
```



```

        v_patient_name,
        ' discharged. Bed is now available.'
    );
END IF;
END IF;
END//

DELIMITER ;

```

Procedure 3: Process Entire Waitlist (Batch Operation) Uses **CURSOR** with **LOOP** to iterate through all waiting patients:

```

DELIMITER //

CREATE PROCEDURE sp_process_waitlist(
    OUT p_allocated_count INT,
    OUT p_remaining_count INT
)
BEGIN
    DECLARE v_waitlist_id INT;
    DECLARE v_patient_id INT;
    DECLARE v_ward_type VARCHAR(20);
    DECLARE v_bed_id INT;
    DECLARE v_done BOOLEAN DEFAULT FALSE;
    DECLARE v_allocated INT DEFAULT 0;

    -- Cursor to iterate through all waiting patients
    DECLARE waitlist_cursor CURSOR FOR
        SELECT waitlist_id, patient_id, requested_ward_type
        FROM waitlist
        WHERE status = 'Waiting'
        ORDER BY priority_score DESC, request_time ASC;

    DECLARE CONTINUE HANDLER FOR NOT FOUND
        SET v_done = TRUE;

    START TRANSACTION;

    OPEN waitlist_cursor;

    read_loop: LOOP
        FETCH waitlist_cursor INTO v_waitlist_id, v_patient_id, v_ward_type;

        IF v_done THEN
            LEAVE read_loop;
        END IF;

        -- Find available bed for this patient's requested ward type
        SELECT b.bed_id INTO v_bed_id
        FROM beds b
        JOIN wards w ON b.ward_id = w.ward_id

```

```

WHERE w.ward_type = v_ward_type
      AND b.current_status = 'Available'
      AND b.equipment_status != 'Broken'
LIMIT 1
FOR UPDATE;

IF v_bed_id IS NOT NULL THEN
    -- Allocate bed
    UPDATE waitlist
    SET status = 'Allocated'
    WHERE waitlist_id = v_waitlist_id;

    INSERT INTO admissions (
        patient_id, bed_id, status, admission_date, notes
    ) VALUES (
        v_patient_id, v_bed_id, 'Active', NOW(),
        'Auto-allocated from waitlist batch process'
    );

    UPDATE beds
    SET current_status = 'Occupied'
    WHERE bed_id = v_bed_id;

    SET v_allocated = v_allocated + 1;
END IF;
END LOOP;

CLOSE waitlist_cursor;

COMMIT;

SET p_allocated_count = v_allocated;

-- Count remaining
SELECT COUNT(*) INTO p_remaining_count
FROM waitlist
WHERE status = 'Waiting';
END//

DELIMITER ;

```

Procedure 4: Calculate Priority Score

```

DELIMITER //

CREATE PROCEDURE sp_update_waitlist_priority()
BEGIN
    -- Update priority scores based on waiting time
    -- Formula: priority_score = base_priority + (hours_waited * 10)
    UPDATE waitlist
    SET priority_score =

```

```

        CASE
            WHEN priority_score < 100 THEN 100
            ELSE priority_score
        END + (TIMESTAMPDIFF(HOUR, request_time, NOW()) * 10)
    WHERE status = 'Waiting';
END//

DELIMITER ;

```

5.3 Views

View 1: Current Occupancy Dashboard

```

CREATE VIEW vw_current_occupancy AS
SELECT
    w.ward_type,
    COUNT(b.bed_id) AS total_beds,
    SUM(CASE WHEN b.current_status = 'Occupied' THEN 1 ELSE 0 END) AS occupied_count,
    SUM(CASE WHEN b.current_status = 'Available' THEN 1 ELSE 0 END) AS available_count,
    SUM(CASE WHEN b.current_status = 'Maintenance' THEN 1 ELSE 0 END) AS maintenance_count,
    ROUND(
        SUM(CASE WHEN b.current_status = 'Occupied' THEN 1 ELSE 0 END) * 100.0 / COUNT(b.bed_id),
        2
    ) AS occupancy_rate_percent
FROM wards w
LEFT JOIN beds b ON w.ward_id = b.ward_id
GROUP BY w.ward_type;

```

View 2: Active Admissions with Full Details Uses JOIN across 5 tables:

```

CREATE VIEW vw_active_admissions AS
SELECT
    a.admission_id,
    p.patient_id,
    p.name AS patient_name,
    p.age,
    p.blood_group,
    p.contact,
    w.ward_type,
    w.floor,
    b.bed_id,
    b.bed_number,
    s.name AS doctor_name,
    s.phone AS doctor_phone,
    a.admission_date,
    a.expected_discharge,
    DATEDIFF(COALESCE(a.actual_discharge, NOW()), a.admission_date) AS days_admitted,
    a.notes
FROM admissions a
INNER JOIN patients p ON a.patient_id = p.patient_id
INNER JOIN beds b ON a.bed_id = b.bed_id

```

```

INNER JOIN wards w ON b.ward_id = w.ward_id
LEFT JOIN staff s ON a.doctor_id = s.staff_id
WHERE a.status = 'Active';

```

View 3: Available Beds with Ward Info

```

CREATE VIEW vw_available_beds AS
SELECT
    b.bed_id,
    b.bed_number,
    w.ward_id,
    w.ward_type,
    w.floor,
    w.daily_charge,
    b.equipment_status,
    w.capacity - (
        SELECT COUNT(*)
        FROM admissions a2
        JOIN beds b2 ON a2.bed_id = b2.bed_id
        JOIN wards w2 ON b2.ward_id = w2.ward_id
        WHERE w2.ward_id = w.ward_id AND a2.status = 'Active'
    ) AS remaining_capacity_in_ward
FROM beds b
JOIN wards w ON b.ward_id = w.ward_id
WHERE b.current_status = 'Available'
ORDER BY w.ward_type, b.bed_number;

```

View 4: Waitlist with Patient Details and Waiting Time

```

CREATE VIEW vw_waitlist_details AS
SELECT
    wl.waitlist_id,
    p.patient_id,
    p.name AS patient_name,
    p.age,
    p.blood_group,
    p.contact,
    wl.requested_ward_type,
    wl.priority_score,
    wl.request_time,
    TIMESTAMPDIFF(HOUR, wl.request_time, NOW()) AS waiting_hours,
    TIMESTAMPDIFF(MINUTE, wl.request_time, NOW()) % 60 AS waiting_minutes,
    CONCAT(
        TIMESTAMPDIFF(HOUR, wl.request_time, NOW()), 'h ',
        TIMESTAMPDIFF(MINUTE, wl.request_time, NOW()) % 60, 'm'
    ) AS waiting_time_formatted,
    wl.status,
    wl.notes
FROM waitlist wl
INNER JOIN patients p ON wl.patient_id = p.patient_id
WHERE wl.status = 'Waiting'

```

```
ORDER BY wl.priority_score DESC, wl.request_time ASC;
```

View 5: Bed Turnover Analysis

```
CREATE VIEW vw_bed_turnover AS
SELECT
    w.ward_type,
    COUNT(a.admission_id) AS total_discharges,
    ROUND(AVG(DATEDIFF(a.actual_discharge, a.admission_date)), 2) AS avg_stay_days,
    ROUND(MAX(DATEDIFF(a.actual_discharge, a.admission_date)), 2) AS max_stay_days,
    ROUND(MIN(DATEDIFF(a.actual_discharge, a.admission_date)), 2) AS min_stay_days,
    ROUND(
        SUM(DATEDIFF(a.actual_discharge, a.admission_date)) / COUNT(DISTINCT a.bed_id),
        2
    ) AS avg_turnover_per_bed
FROM admissions a
JOIN beds b ON a.bed_id = b.bed_id
JOIN wards w ON b.ward_id = w.ward_id
WHERE a.status = 'Discharged'
    AND a.actual_discharge IS NOT NULL
    AND a.admission_date >= DATE_SUB(NOW(), INTERVAL 30 DAY)
GROUP BY w.ward_type;
```

View 6: Doctor Workload

```
CREATE VIEW vw_doctor_workload AS
SELECT
    s.staff_id,
    s.name AS doctor_name,
    s.department,
    s.phone,
    COUNT(a.admission_id) AS current_patients,
    GROUP_CONCAT(DISTINCT w.ward_type) AS assigned_wards
FROM staff s
LEFT JOIN admissions a ON s.staff_id = a.doctor_id AND a.status = 'Active'
LEFT JOIN beds b ON a.bed_id = b.bed_id
LEFT JOIN wards w ON b.ward_id = w.ward_id
WHERE s.role = 'Doctor'
GROUP BY s.staff_id, s.name, s.department, s.phone;
```

5.4 Indexes

```
-- Bed status lookups (frequent in allocation queries)
CREATE INDEX idx_bed_status ON beds(current_status);

-- Combined index for ward + status (dashboard filtering)
CREATE INDEX idx_bed_ward_status ON beds(ward_id, current_status);

-- Admission status filtering (discharge panel, active list)
CREATE INDEX idx_adm_status ON admissions(status);
```

```

-- Active admission per bed (constraint enforcement)
CREATE INDEX idx_adm_bed_active ON admissions(bed_id, status);

-- Patient name search
CREATE INDEX idx_patient_name ON patients(name);

-- Patient contact lookup (unique constraint already creates index, but explicit for clarity)
-- CREATE INDEX idx_patient_contact ON patients(contact); -- Not needed, UNIQUE creates implicit index

-- Waitlist sorting by priority and time
CREATE INDEX idx_waitlist_priority ON waitlist(priority_score DESC, request_time ASC);

-- Waitlist status filtering
CREATE INDEX idx_waitlist_status ON waitlist(status, requested_ward_type);

-- Audit log lookups by bed
CREATE INDEX idx_audit_bed ON bed_status_audit(bed_id, changed_at DESC);

-- Maintenance log by bed
CREATE INDEX idx_maint_bed ON bed_maintenance_log(bed_id, start_time DESC);

```

5.5 Assertions (Implemented via Triggers & CHECK Constraints)

MySQL 8.0 does not support the SQL standard `CREATE ASSERTION` statement. The following business rules are enforced using **CHECK constraints** and **triggers** as MySQL-compatible equivalents:

Assertion Rule	Implementation
A bed cannot have more than one active admission	Trigger <code>trg_prevent_double_active_admission</code> (BEFORE INSERT)
Maintenance cannot start on an occupied bed	Trigger <code>trg_prevent_maint_occupied</code> (BEFORE INSERT)
Ward capacity cannot be exceeded	Application-level check + Trigger (count active admissions ward.capacity)
Expected discharge must be after admission date	CHECK constraint <code>chk_adm_dates</code>
Bed status must be valid	CHECK constraint <code>chk_bed_status</code>
Patient age must be realistic	CHECK constraint <code>chk_patient_age</code>
Staff role must be valid	CHECK constraint <code>chk_staff_role</code>

Trigger: Enforce Ward Capacity

```

DELIMITER //

CREATE TRIGGER trg_enforce_ward_capacity
BEFORE INSERT ON admissions
FOR EACH ROW
BEGIN
    DECLARE v_capacity INT;
    DECLARE v_current_occupied INT;
    DECLARE v_ward_id INT;

```

```
SELECT w.ward_id, w.capacity INTO v_ward_id, v_capacity
FROM beds b
JOIN wards w ON b.ward_id = w.ward_id
WHERE b.bed_id = NEW.bed_id;

SELECT COUNT(*) INTO v_current_occupied
FROM admissions a
JOIN beds b ON a.bed_id = b.bed_id
WHERE b.ward_id = v_ward_id AND a.status = 'Active';

IF v_current_occupied >= v_capacity THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Error: Ward capacity exceeded. Cannot admit more patients.';
END IF;
END//

DELIMITER ;
```

6. Transaction Design

6.1 Isolation Levels

Operation	Isolation Level	Reason
Bed Allocation	REPEATABLE READ	Prevents phantom reads when checking available beds
Discharge	REPEATABLE READ	Ensures admission record doesn't change mid-operation
Waitlist Batch	SERIALIZABLE	Strictest isolation for batch operations
Read-Only Reports	READ COMMITTED	Allows concurrency, sufficient for analytics

6.2 Transaction Patterns

```
-- Pattern for Bed Allocation (used in sp_allocate_bed)
START TRANSACTION;
-- 1. SELECT ... FOR UPDATE (lock available bed row)
-- 2. INSERT INTO admissions
-- 3. UPDATE beds SET current_status = 'Occupied'
-- 4. UPDATE waitlist (if applicable)
COMMIT;
-- ROLLBACK on any error
```

```
-- Pattern for Discharge (used in sp_discharge_patient)
START TRANSACTION;
-- 1. SELECT admission, bed, ward details (verify active)
-- 2. UPDATE admissions SET status = 'Discharged', actual_discharge = NOW()
```

```
-- 3. UPDATE beds SET current_status = 'Available' or 'Maintenance'
-- 4. INSERT INTO bed_maintenance_log (if maintenance)
-- 5. SELECT FROM waitlist (cursor)
-- 6. If match: UPDATE waitlist + INSERT new admission + UPDATE bed
COMMIT;
-- ROLLBACK on any error
```

7. Data Integrity Summary

Constraint Type	Count	Tables Applied
PRIMARY KEY	8	All tables
FOREIGN KEY	10	beds, admissions, waitlist, maintenance_log, audit
UNIQUE	4	wards, beds, patients, waitlist
CHECK	18	All tables (domain constraints)
NOT NULL	15	Critical fields
TRIGGER	6	beds, admissions, bed_maintenance_log
INDEX	10	Performance optimization

8. Seed Data

```
-- Insert Wards
INSERT INTO wards (ward_type, floor, capacity, daily_charge) VALUES
('General', 1, 10, 500.00),
('ICU', 2, 5, 2500.00),
('Emergency', 1, 8, 1000.00),
('Private', 3, 6, 1500.00);

-- Insert Beds (20 total)
INSERT INTO beds (ward_id, bed_number, equipment_status, current_status) VALUES
(1, 'G-101', 'Standard', 'Available'),
(1, 'G-102', 'Standard', 'Available'),
(1, 'G-103', 'Standard', 'Occupied'),
(1, 'G-104', 'Standard', 'Available'),
(2, 'ICU-01', 'Ventilator, Monitor', 'Available'),
(2, 'ICU-02', 'Ventilator', 'Occupied'),
(2, 'ICU-03', 'Monitor', 'Available'),
(2, 'ICU-04', 'Ventilator, Monitor', 'Maintenance'),
(2, 'ICU-05', 'Standard', 'Available'),
(3, 'E-101', 'Oxygen', 'Available'),
(3, 'E-102', 'Oxygen, Monitor', 'Occupied'),
(3, 'E-103', 'Standard', 'Available'),
(3, 'E-104', 'Oxygen', 'Available'),
(4, 'P-301', 'Premium', 'Available'),
```



```

(4, 'P-302', 'Premium', 'Occupied'),
(4, 'P-303', 'Premium', 'Available'),
(4, 'P-304', 'Premium', 'Available'),
(4, 'P-305', 'Premium', 'Maintenance'),
(4, 'P-306', 'Premium', 'Available');

-- Insert Staff
INSERT INTO staff (name, role, department, phone) VALUES
('Dr. Rajesh Sharma', 'Doctor', 'Cardiology', '9876543210'),
('Dr. Priya Patel', 'Doctor', 'General Medicine', '9876543211'),
('Dr. Amit Verma', 'Doctor', 'Emergency', '9876543212'),
('Dr. Sunita Rao', 'Doctor', 'ICU', '9876543213'),
('Nurse Anjali', 'Nurse', 'ICU', '9876543214'),
('Nurse Kumar', 'Nurse', 'General', '9876543215'),
('Admin Singh', 'Admin', 'Administration', '9876543216');

-- Insert Patients
INSERT INTO patients (name, age, blood_group, contact, address) VALUES
('Amit Kumar', 45, 'O+', '9123456789', 'Delhi'),
('Sunita Devi', 32, 'B+', '9123456790', 'Mumbai'),
('Rahul Singh', 67, 'A-', '9123456791', 'Bangalore'),
('Priya Nair', 28, 'O+', '9123456792', 'Chennai'),
('Vikram Shah', 55, 'B-', '9123456793', 'Pune'),
('Anita Gupta', 41, 'AB+', '9123456794', 'Kolkata');

-- Insert Active Admissions
INSERT INTO admissions (patient_id, bed_id, doctor_id, expected_discharge, status, notes) VALUES
(1, 3, 2, '2026-05-20', 'Active', 'General checkup'),
(2, 6, 4, '2026-05-18', 'Active', 'Post-surgery monitoring'),
(3, 12, 3, '2026-05-19', 'Active', 'Emergency trauma'),
(4, 15, 1, '2026-05-22', 'Active', 'Private room recovery');

-- Insert Waitlist
INSERT INTO waitlist (patient_id, requested_ward_type, priority_score, status) VALUES
(5, 'ICU', 500, 'Waiting'),
(6, 'General', 200, 'Waiting');

-- Insert Maintenance Log
INSERT INTO bed_maintenance_log (bed_id, reason, status, resolved_by) VALUES
(8, 'Ventilator servicing', 'Ongoing', 4),
(17, 'Deep cleaning', 'Ongoing', 5);

```

End of Backend Schema Document